Selling a Place Once: Ticketing a Venue on a Replicated Log

Oscar Martinez
MOG Systems, Inc.

Abstract

A ticketing system for a large venue must sell each place at most once, keep a party together, keep selling and admitting at a site that loses its line to the centre, and do all of it when thousands of parties ask for one window in the same second. A common design buys these guarantees with a database lock per place, a queue in front of it, and a separate system at the gate that is reconciled after the fact. This paper shows that the state primitive of the companion paper [1] carries the whole domain without a second system. A place in one window and one epoch has one deterministic id, so one subject on one stream, and its first version is published expecting that subject to be empty; the stream's leader admits one such write and refuses every other, for as long as the stream keeps what it acknowledged. A party's claim on a window is one record at one position of one stream, and the verdict on it reads only that stream's prefix, so the party wins every place it asked for or none. A site verifies a signed token and records the entry on its own stream while cut off, and the hub's stream takes the site's records in the site's order when the line returns. Every check stays the bounded local evaluation of the companion. The parts are known: an expected empty write, a deterministic executor over an ordered log, a signed ticket checked offline. The contribution is their composition under one naming, with its proofs and its boundaries. I give the records, the contended write with its proof and the conditions the proof needs, the changes a sold place goes through, the arithmetic of a park day against the companion's measured latencies, the split between what a site owns and what only the hub may write, and the boundaries of the claim. The venue tenant this paper describes is a design and a record family specification on the primitive as built, not a running deployment; every measured number is the companion's, and every other number is derived with its arithmetic shown.

1. The problem

1.1. What a ticketing system must guarantee

The venue I design for is a complex at the scale of a theme park. A film screening is one attraction in it. Around the screening run rides and shows with timed entry windows, restaurants with VIP tables sold by seating, a merch store, bars and kitchens, and concession carts that move through the park. A guest arrives as part of a party. A party's day is a set of claims on windows that must not overlap, and some windows admit a party only if its members' ages meet a rule.

Four guarantees follow, and a system that misses any one of them fails in front of a guest.

1. **No place is sold twice.** Two parties after the last four seats of a screening must not both get them, whatever the timing of their requests and however many servers handle them.
2. **A party is held whole.** A party of four gets four places in a window or none. It never gets three, and it never gets a ride that overlaps another of its rides.
3. **The gate keeps working.** A park whose line to the centre goes down, or a cart that rolls into a dead zone, keeps admitting valid tickets and taking sales, and what it did reaches the centre when the line returns.
4. **A popular window's opening is carried.** A popular window opens and thousands of parties ask for it in the same second.

1.2. What a common design pays for them

A common design takes three systems. An inventory database holds each place as a row and takes a lock on it, or uses a conditional update, to sell it once. A waiting room in front of the storefront meters arrivals so that the database is not asked more than it can answer. The gates run on a separate access control system with its own copy of the ticket list, synchronised in batches, and a reconciliation job afterwards finds what the copy missed.

Each boundary costs something. A lock per place is a network round trip to the database on the write path, and a request for four places is a transaction over four rows. The gate's copy is a second source of truth, so a refund at the centre and an entry at the gate have no common order and no common record, and the reconciliation compares two systems' reports. A large on sale is fought in the waiting room, which meters what reaches the inventory: in the best documented failure, the seller slowed its queues on purpose to keep its systems up [6].

1.3. The claim

The four constraints of the companion carry this domain without a second system. A place is sold at most once by construction: one id, one subject, and the broker's expected empty publish, on a stream set to keep what it holds (section 3.3). A party is held whole or not at all, because the verdict on its claim is one deterministic function of one stream's prefix. A site scans while cut off and the hub converges when the line returns, by the companion's site and its fold. And every check on every write is the bounded local evaluation of the companion, with no call to another service.

None of these parts is new. A first write that expects an empty key is how the broker's own key value store creates a key [13], and how event stores keep a value unique [16], [17]. A sequencer followed by a deterministic executor is Calvin's [9]. A signed ticket checked offline and uploaded later is an existing design for transit validators [18]. What this paper adds is their composition on one log, under one naming for the type, the route and the grant, with the proof that the composition holds (section 7).

The claim is no stronger than that. The waiting room stays in front of the storefront, because fairness among people is not an order among writes (section 6). A site that is cut off cannot know of a refund made at the hub after the cut (section 5.2). The guarantee rests on the broker keeping what it acknowledged, which its default flush to disk does not promise (section 3.3). The rate is derived, not measured, from writes the broker acknowledged before it flushed them, and section 4 names every assumption the derivation makes.

2. The records and the key

2.1. The subject

Every record is published to one subject of five tokens (companion section 2.2):

```
tenant . <tenant> . dag . <family> . <id>
```

The family is `Kind(schema_id)`, a total function of the record's schema identifier, so the writer does not choose it (companion section 2.2). The id is either content addressed or derived from the fields that define the record: SHA-256 over the fields joined by a zero byte, truncated to 15 bytes and encoded in base64url, 120 bits in 20 characters, always one subject token (companion section 2.1). A family has at most one stream per tenant, bound to its branch of the tree. A family that a tenant's kind does not carry has no stream under that tenant, so a record of it has nowhere to land. Evaluating a write is the companion's fixed sequence of local lookups on that one string: derive the subject, match the grant, find the stream, validate the record. The work is bounded by a constant that depends on the grant list, the stream table and the schema, plus validation work in the size of the record, and nothing in it depends on the number of records, tenants or identities stored (companion section 3.2). Every write this paper makes, a hold, a reservation or a scan, passes that evaluation and no other check before the broker. At the broker the writer chooses the subject it sends, within its grant. After the log, the projector folds a record only at its own address: exactly five tokens, `tenant` first and `dag` third, the tenant of the stream it was read from, `Kind(schema_id)` in the family token and the record's own id in the last, on a stream of that family. It refuses any other subject, so a record published off its address never reaches the read model (companion sections 2.2 and 3.4). The check guards the read model, not the log, and the guarantee of section 3 rests on the broker's expectation, not on it.

The venue is one tenant, of a fourth kind beside the creator, the product and the store, the three kinds the studio paper describes [27]. A tenant is its tenant record on the platform root, which names its kind, and the bootstrap creates one stream under the tenant's root for each family that kind carries, with no build and no deploy (companion sections 2.1 and 5); this holds in the reference implementation. A family arrives as a schema

record on the log, naming the family and the tenant kinds that carry it, and the bootstrap creates the family's stream under each recorded tenant of those kinds (companion sections 2.3 and 5); this also holds in the reference implementation. The kinds a tenant record or a schema record may name are a closed list in those two schemas, creator, product and store, and a schema is immutable per identifier. The kind venue is therefore a new identifier of each of those two schemas and an entry in the list the bootstrap compiles: a build of the primitive, which this paper takes as part of the design. The venue's families are the specification of section 2.2; none of them is on the log.

2.2. The families

The venue carries 45 families. The set is a specification: the venue has no tenant record and no schema record on the log, and what follows is what those records would carry. Fifteen every kind carries: decisions, jobs, runs, exceptions and the rest of the ledger's own records, and the marketing families a venue uses to promote its days, the shared set of the studio paper [27]. Twelve it shares with the store kind's specification, whose main families the studio paper names [27], because a bar's sale is the same record as a store's sale: orders, stock moves, products, bills of materials (for a bar, its recipes), suppliers, purchase orders, shipments, customer service tickets, campaigns, affiliates, payouts and documents. Eighteen are the venue's own. These are the ones this paper uses:

Family	One record is	Id over
`space`	One place with its own capacity and rules: an attraction, a screen, a restaurant, one VIP table	the venue and the space's key
`window`	One timed slot of one space: a ride's fifteen minutes, a screening, a table's seating	the space and the start time
`party`	One group that must be held together: its size and its members' age bands	the storefront's booking id
`hold`	One party's claim on one window, settled whole or refused whole	the party and the window
`reservation`	The one winner of one place in one window in one epoch	the window, the place's key and the epoch
`admission`	One person's right to enter one window: what the public calls the ticket	the reservation
`voucher`	One prepaid right to redeem: a drinks package, a meal plan, parking	the order, its line and an index
`scan`	One accepted read of an admission or a voucher	the admission or voucher, the purpose, the epoch and the site

Each id is derived over the tenant and the family as well as the fields shown. The others are the venue, events, ticket types, invitations, guest lists, promotions, offers, concession stands, menu items and settlements. Edges join them: a hold to its party and its window, a reservation to the hold that won it, an admission to its reservation, a scan to what it admitted, a table's order to the table's hold.

2.3. Places are keys, not records

A seat is a key from the seat map. A place in a window that is not seated is a number from 1 to the window's capacity. A VIP table is one place, the table. None of these is written as a record until it is won. A full park day holds about 655,000 places (section 4.4), and writing each as a record before it sells costs writes and storage that no reader needs. Only a won place becomes a record, its reservation, and the reservation's id is the place. The guarantee of section 3 is per key. That two keys of a seat map name two physical seats, and that a layout changed after a window goes on sale keeps each sold seat under its key, is the seat map's to keep, not the log's.

2.4. Nobody is named on the log

The log is append only and replayed, so nothing on it can be deleted on request. A buyer's name, email, card and address stay with the payment processor. A party is a size and its members' age bands, never who they are. An age is a band, `under_3`, `3_6`, `7_12`, `13_17`, `18_20` or `21_plus`, with the reference of the check that verified it, never a birth date or what an identity document said. A stand's customer is a card in the point of sale system, which the log never sees. No card number reaches the log, the hub or a site: the processor's connector drops every personal

field before it writes, and a stand's terminal keeps the card and sends its site only the sale. The card industry's security standard applies to the systems that store, process or transmit card data [21]. The design keeps the log, the hub and the sites outside that set, and keeping them out is the connectors' work, so it is a property of the design to be verified when the connectors are built. A party's key is the storefront's booking id, and that id leads to a buyer through the processor's records. Whether a record that can be linked back that way counts as personal data is a question for the law where the venue runs; the design keeps the link outside the log, where it can be cut.

2.5. Who writes what

A grant is a set of tuples, an identity, a verb and a pattern over the subject tree, where the verb is publish or subscribe and the pattern is a subject in which a token may be * or which may end in > (companion section 2.4). The venue's members are memberships, an identity, the tenant and a role, and from them the bootstrap emits the broker's grants; a grant is a configuration change that each broker reloads in place, one naming with the streams and the registry but not one store (companion sections 2.6 and 5). Every identity receives its replies on its own reply prefix, which the rendering appends to its subscribe list, and no other identity can read them (companion section 2.4). That covers what a member asks of the broker, such as a site worker's requests across the leaf link. A caller reaches the authority through the web service in front of it, which sets the tenant from the caller's session or key on every route [27], and is answered there. I hold the manager's membership, which writes everything and settles every decision, exception and settlement. Guest services, the box office, attraction leads and stand leads write invitations and guest lists, ask for house holds, and write the counts at shift close. The processor's connector writes parties and orders, and asks for a hold's conversion when a payment clears. The point of sale connector writes the sales that reach the hub from the point of sale vendor's cloud. Each site's worker writes that site's scans, sales, stock moves and shift counts. The venue's agent, for model jobs, writes drafts and proposals. The authority writes the settled families itself: holds' verdicts, reservations, admissions and vouchers. No member of the venue holds a grant on them, so the check of companion section 3.1 refuses a member's reservation before the broker routes it. The identities that hold a tenant's whole tree are the companion's authority, projector and emergency identity (companion section 2.4), and of these only the settler, inside the authority, writes reservations. A read of the read model is bound to one tenant by the service that binds the tenant from the caller's credential, the row policy makes a read that binds no tenant fail, and a read of another tenant's row answers as a read of a missing row does (companion sections 2.5 and 3.4). A service that bound the wrong tenant would read that tenant's rows, so the line between tenants is the binding service's, with the policy as its backstop. The tenant is the one line the venue's read policy draws: every member reads every record of the venue, and nobody is named on the log for a finer line to guard (section 2.4).

3. The contended write

3.1. The hold

A caller asks the authority to hold a window for a party. The caller is the storefront through the processor's connector, a guest services desk, or me. The authority writes the hold with the state `placed`, a round number, an expiry, and a copy of everything the verdict reads from other families: the party's members and their bands, the window's start, end and capacity, the space's age rule and covers, the ticket type the window requires, and the seats asked for when the window is seated. A request repeated in the same round has the same id, and the authority answers it with the hold already there. Every input the authority appends to a hold, the placed version first, expects the last sequence it read on the hold's subject, as every version the settler appends does (section 3.2). Two copies of the authority that place the same round therefore write one placed version; the broker refuses the other, and its writer reads the one there. The authority appends a conversion or a release only to a hold whose verdict it has read. A request for a window the settler already knows is full is refused at once, with no write.

The hold's id is the party and the window. A party that tries the same window again after losing, releasing or letting it expire writes the next round on the same id. A hold is never retracted: a retracted id takes no edge again (companion section 6.5), and every later round needs its edges to its order and its table's orders. Expiry, release and a lost verdict are states, and the round says which attempt is current.

3.2. The settler

The settler is one consumer in the authority, per tenant, reading the tenant's hold stream in stream order. The stream carries two kinds of hold version. Inputs are the versions that change what is held: `placed`, `converted` and `released`, which the authority appends on a caller's request, and `expired`, which the settler appends when its clock passes a hold's expiry. Verdicts are `won` and `lost`, which the settler appends to decide a placed hold. In memory the settler keeps each place's current epoch and holder, each window's free count, and each party's won windows for the day. An input takes effect in memory at its own position in the stream, when the settler reads it, never earlier; an expiry the settler has decided takes effect only when it reads its own `expired` version back. A verdict takes effect at the position of the hold it decides. At start the settler rebuilds its memory the same way: each input at its position, and each recorded verdict at the position of its hold. For each placed hold it checks, in order:

1. **Age.** Every member's band meets the space's age rule, and any band that needs an adult has one in the same party. Else `lost` with the reason `age_rule`.
2. **Requires.** The party has won the ticket type this window requires, such as the park day. Else `requires`.
3. **Overlap.** The window does not overlap, or come within the walking gap of, a window the party has won that day. Else `party_overlap`.
4. **Size.** For a table, the party's size is inside the table's covers. Else `covers`.
5. **Places.** One free place per member: the seats asked for or the best available (section 3.6), the lowest free numbers in a pool, or the table. Else `taken`, `no_adjacent` or `full`.

If every check passes, the settler publishes one reservation per member, each with the state `held` and each expecting an empty subject, and then the hold's `won` version. If any check fails, it publishes the hold's `lost` version with the reason, and nothing else. The authority answers the caller with the verdict.

Every version the settler publishes, of a hold or of a reservation after its first, is published expecting the subject's last sequence to be the one the settler read. Two settlers therefore cannot both append the next version of one record; the broker refuses the second, and the refused one reads what is there. The settler gives each message an id derived from the record's id and content, so a repeated publish of the same version is one the broker can recognise.

A snapshot. Rebuilding from the start of the hold stream grows with the season. The companion's replay read 1,454,979 messages in 117 seconds (companion section 6.7). At that rate a season of a hundred park days of section 4.4, up to 36 million hold versions, takes about 48 minutes to read. So the settler writes a snapshot of its memory at a stream position up to which every verdict is published. It writes one every 10,000 hold versions. The snapshot holds only what an open window needs: each open place's epoch and holder, each open window's free count, and each party's won windows for the open days. It is a file in the tenant's bucket, named on the log by a document record whose id is the hash of its bytes and which names the position. At about 64 bytes a place, a full park day's 655,000 places is about 42 MB. A settler that starts loads the newest snapshot and reads the hold stream from its position: at most 10,000 versions, under a second at the replay's rate. The snapshot's own load time was not measured. By Lemma 3 the memory at a position is a function of the stream before it, so a snapshot is the memory any settler would rebuild there. The check named `snapshot_matches` rebuilds the memory from the stream to each snapshot's position once a day and compares the two.

3.3. The guarantee

The broker gives one property this design rests on. A publish may carry a header that names the last sequence it expects on its subject. The header is `Nats-Expected-Last-Subject-Sequence`; the Go client sets it with `WithExpectLastSequencePerSubject` and documents it as optimistic concurrency control at the level of the subject [2]. The stream's leader stores the message only if the last message on that subject has the sequence named, and with the value 0, only if the subject holds no message. The companion's schema registry already depends on this: of two writers racing to the first append of a schema, the broker refuses the second, because the publish expects an empty subject (companion sections 2.3 and 6.2).

The expectation reads what the stream holds, so what the stream holds must not shrink, and it does not: a record is never dropped. Every record stream at the hub discards new messages at its cap rather than old ones, and denies delete and purge [2], so a stream at its cap refuses the next write with an error at the writer and keeps everything it holds. A fill metric per record stream raises an alarm at 80 percent of the cap. A write also fails on the storage a broker uses, not on what the caps reserve, so each broker's storage used is measured against its limit, and an alarm is raised when the fullest broker reaches 70 percent or the measure goes missing, before any write fails on it (companion section 2.2). The venue design gives every stream at its sites the same three settings, so that a site's scan stream at its cap refuses the next entry rather than forgetting an earlier one. No alarm of the hub's watches a site's streams, so a site's scan stream is sized for every record of every open window (section 4.4). The single sale needs exactly these settings. A stream that discarded old messages to make room would empty a reservation's subject, and the broker accepts an expected empty write again on a subject whose messages were purged [13]. The reservation and hold streams need one thing more: a cap set from the venue's figures, large enough for every reservation of every open window (section 4.4). The stream itself can still be deleted by the identity that administers streams, and a deleted stream holds nothing; section 8 states this as a boundary.

Write *r(w, u, e)* for the reservation of place u in window w in epoch e. Its id is derived from the tenant, the family, w, u and e, so it is one subject, and the family function puts that subject on one stream, the tenant's reservation stream at the hub.

Lemma 1 (one first version per place and epoch). The reservation stream holds at most one first version of *r(w, u, e)*.

Proof. The id is a function of *(w, u, e)*, so every first version of *r(w, u, e)* is published to one subject, and the family function sends that subject to one stream (companion section 2.2). The stream is totally ordered by its own Raft group, and the side of a partition without a majority cannot write at all (companion section 6.1), so there is one sequence of messages on the subject, decided by one leader at a time. Every first version is published expecting the subject empty. The first one the leader stores makes the subject non-empty, and while it is retained every later one that expects it empty is refused. With these settings, the reservation stream retains every reservation of a window until the window closes. ■

What the lemma assumes of the broker. The proof takes a stored message to stay stored, and one leader at a time. The broker acknowledges a write once a majority of the stream's replicas hold it, but by default a server flushes to disk only every two minutes. Jepsen's analysis of nats-server 2.12.1 lost about thirty seconds of acknowledged writes to simulated power failures, and with process pauses and one simulated power failure saw a cluster fall into a split brain that persisted after the network healed [15]. A lost first version empties its subject, and a split brain is two leaders; either admits a second winner. A server on the default interval is exposed to the first, and the companion states the boundary that follows: acknowledged writes can be lost if every replica of a stream loses power within the flush window (companion section 6.1). This paper has not tested the cluster for either fault. So Lemma 1 holds while the broker keeps what it acknowledges and elects one leader at a time. The venue design requires each server to flush every write before it acknowledges it, the setting the companion names for a tenant that needs durability against simultaneous power loss, and no measurement in section 4 used that setting.

Lemma 2 (one live epoch). At every position of the reservation stream, at most one epoch of a place is live, where live is `held`, `sold`, `comped` or `allotted`.

Proof. A place's epoch rises only when its reservation is released. Before it publishes the first version of *r(w, u, e + 1)*, the settler requires that the last message on the subject of *r(w, u, e)* is that reservation's `released` version, from its own acknowledgement or, after a restart, by reading the subject. This read guards the write; it does not enter the verdict. If the guard fails, because the release was refused or is not yet acknowledged, the settler retries the release and publishes no verdict until it is through. A verdict exists only once it is published, so a settler that waits contradicts no recorded verdict: the hold waits, and a stream, or a broker, too full to take the release stops sales rather than doubling one. A released reservation takes no later version: the settler writes none after `released`, and any version another settler might append expects a last sequence older than the release, so

the broker refuses it. By induction on the positions of the stream, when epoch *e + 1* first appears, epoch *e* is released and stays released. ■

Theorem (a place is sold at most once at a time). At every position of the reservation stream, each place has at most one live reservation, and so at most one party holding it, sold or not.

Proof. By Lemma 2 at most one epoch of the place is live, and by Lemma 1 that epoch has one reservation. A sale is a state of that reservation. A sold reservation is released only by a refund, an exchange or an upgrade, and each voids its admission (section 3.6). ■

Lemma 1 is the broker's guarantee and, under those conditions, holds however many writers race, including two copies of the authority. Lemma 2 rests on one rule in the settler, the release before the next epoch. Neither rests on a lock held in a process, on a clock, or on the settler being the only one running. The check named `one_winner` reads the reservation stream back and confirms that no place has two live reservations; by the theorem it never fails, and it is the proof read back. It reads through the companion's address check, so a record off its own address is the reservation of no place, and the projector raises it as an exception (section 2.1).

The guarantee holds for every publish that carries the expectation. The grants keep every member off the reservation family (section 2.5), so the remaining trust is in one rule of one program: the settler sets the header on every first version. Section 8 states this as a boundary.

3.4. A party is held whole

Lemma 3 (one verdict). The settler's verdict on a placed hold at position p of the hold stream is a function of the inputs before p and the hold itself.

Proof. By induction on the placed holds in stream order. Every value the five checks read is in the hold, in the settler's memory, or in a seat map the hold names by the hash of its bytes, which every reader reads alike (section 3.6). The hold carries its members, its window's times and capacity, its space's rules and the ticket type it requires, copied when it was placed (section 3.1). The memory at p is the effect of every input before p, each applied at its own position, and of the verdict on every placed hold before p, each applied at its hold's position (section 3.2). By the induction hypothesis each of those verdicts is a function of the inputs before its hold, so the memory at p is a function of the inputs before p. The reservations are consequences of verdicts and are never read to decide. So the verdict is a function of the inputs before p and the hold. ■

The settler's clock decides when an expiry is appended, and so where it lands. Once it has landed it is an input like any other, and every settler and every replay applies it at that position. The clock chooses the input; it never makes two readers of the stream disagree about it.

Copying the window's and the space's fields onto the hold is what makes the lemma true. A settler that read capacity or an age rule from its own copy of the window or space stream would decide on whatever version of that stream it had seen, and two settlers, or a settler and a replay, could reach different verdicts. Nothing orders a message on one stream against a message on another (companion section 6.5), so the settler reads one stream.

By Lemma 3 every settler and every replay reaches the same verdict on the same hold, for the whole party. The places are separate reservations on separate subjects. On broker versions before 2.12, the log does not make them land as one write, and a reader of the read model can see three of a party's four reservations for a moment. From version 2.12 a stream can store a batch of messages whole or not at all, checking each message's expectation on its own subject [14], so a party's reservations, which are all on the reservation stream, could land as one batch. The hold's `won` version is the commit point either way, and it is on the hold stream, so a batch would not move it. It is published only after every reservation of the hold is acknowledged, and everything that acts on a hold, conversion first among them, reads the hold, never the reservations.

A settler that stops halfway. Its replacement reads the same stream, reaches the same verdict by Lemma 3, and publishes the same reservations under the same ids with the same content. Within the stream's duplicate window, two minutes by default [3], the broker answers the repeated message id as a duplicate and stores nothing. Beyond it, the publish expecting an empty subject is refused. The settler then reads the subject's last message, and if it names the same hold and round, the reservation is the settler's own and counts as written. If it names another hold, the place was taken by a writer the settler did not know of, which the grants exclude. The settler

then releases what it wrote for the hold and publishes `lost`, so the party is still never partly held. That `lost` is not the function of Lemma 3, because it answers a write the grants exclude, but it is recorded on the hold stream, and a settler that rebuilds applies each recorded verdict at its hold's position (section 3.2), so every settler still agrees. The check named `whole_parties` reads the outcome back: every won hold has one reservation per member, and no reservation names a hold that lost.

Two settlers at once. During a deploy two instances of the authority run side by side for minutes. Both read the same hold stream and reach the same verdicts. For each reservation one of their publishes is stored and the other is refused or answered as a duplicate. For each hold version one is stored and the other is refused, because both expect the same last sequence on the hold's subject. The refused instance reads what is there and finds its own decision. An in process lock could not give this: two processes hold two locks. The broker's expectation is checked by the one stream that holds the subject, however many processes publish to it.

3.5. Conversion, expiry and a payment that clears late

When the payment clears, the processor's connector asks the authority to convert, and the authority appends the hold's `converted` version on the hold's own subject, as an input. The settler reads it in stream order. If the hold is still won in that round, it publishes each reservation's `sold` version and one admission per member, each carrying that member's band and a signed token (section 5.2).

When a hold's expiry passes, the settler publishes its `expired` version and each reservation's `released` version. The settler's clock decides when the expiry is written. It does not decide a race: expiry and conversion are both versions on the hold's subject, so the stream decides which came first, and neither the guest's clock nor the authority's enters. The settler's `expired` version expects the last sequence it read on the subject, so a conversion that landed first makes the broker refuse the expiry, and the settler reads the conversion. If conversion came first, the sale stands. If expiry came first, the authority places the same party in the same window again in a new round, as an input. If the places are still free and the party still passes, the sale stands. If not, the settler raises an exception that names the order, and the payment is refunded in the processor. The log never moves money.

Every claim on capacity goes through the same settler. A house hold that takes places off sale, an invitation's places, a guest list's allowance, a promotion's use cap and a site's door allotment are holds of the same shape, so nothing reaches past a window's capacity by a side door.

3.6. Refunds, exchanges, transfers and accessible seats

Every change to a sold place is an input on its hold's subject, so the hold stream orders it as it orders a sale, and the guarantee of section 3.3 covers it.

A refund. The processor's connector turns a refund into a release of the hold, which the authority appends as an input (section 3.1). The settler publishes each reservation's `released` version and each admission's `void` version, and the place's next sale is its next epoch. A refund of one member names that member's slot. The settler releases that one reservation and voids that one admission, and the hold stays won for the rest of the party. A chargeback reaches the connector as the processor's event and is written as a refund. If the guest has already entered, the entry scan stays on the log beside the void admission, as the venue's record of the entry. The log never moves money (section 3.5).

A place refunded on its window's day. A site that is cut off admits a refunded ticket, because the void has not reached it (section 5.2). If the hub sold that place again, the two tickets would enter as two people in one place. So when a refund lands on the day of its window, the authority places a house hold on one place of that window in the same step, as an input, and releases it when the window closes. The refunded ticket can still enter at a cut off site, but it enters into its own place. A refund on an earlier day is sold again at once.

An exchange or an upgrade. A move to another window or a better seat is a new hold of kind `upgrade` on the new place. When it converts, the settler releases the old reservation and voids its admission. If it loses, the party keeps what it had.

A transfer and a reissue. A token names no one, so a ticket passes to another person as its token does (section 2.4). A transfer that must take the ticket away from its first holder is a reissue. The authority writes the

admission's next version, with a new serial and a new signature, and a site's worker refuses a serial older than its mirror's. A token reissued for a lost wristband is the same step. The entry scan's id is over the admission, not the serial, so at a cut off site the old token and the new one enter once between them in each entry epoch. A resale through the venue's own storefront is a reissue paid through the processor, so the seller's token stops entering wherever the mirror is current. A resale elsewhere is a transfer of a bearer token, which the venue does not see, and the venue's answer to it is the reissue the buyer can ask for at guest services.

Accessible seats. Accessible seats need no new mechanism. An accessible seat and its companion seats are a ticket type of their own, which in the venue design is one price and one allocation, and a house hold keeps them off general sale. The buyer states the need at the storefront. The law may let the storefront ask for that statement, but not demand proof [20], and the log carries the ticket type, never the need. A party asks for the accessible seat and up to three contiguous companion seats in the same row as one hold, so the settler gives all of them or none. The venue releases the house hold when the window's other seats are all sold, or those of its section or its price category, as the rule allows. The settler sees that moment in its own memory, in stream order, and appends the release as an input, as it appends an expiry, so every settler and every replay releases at the same position. An accessible seat is transferred like any other seat.

Choosing seats. A hold on a seated window either names its seats or asks for the best available. A hold that names its seats wins them all or loses as `taken`. A hold that asks for the best available carries the id of the window's seat map, a document whose id is the hash of its bytes (section 2.3), so every settler that opens it reads the same map, and Lemma 3 holds. The map lists the rows in the venue's order of preference, each row's seats in order, and the seats that are accessible. For a party of n, the settler takes the first row in that order with n adjacent free seats whose taking leaves no single free seat between the block and a held seat or the row's end. If no row has such a block, it takes the first row with n adjacent free seats. If no row has n adjacent free seats, the hold loses as `no_adjacent`, unless the party accepted seats apart, in which case the settler gives the n best free seats in the map's order. Accessible seats are never given to a hold that did not ask for one. The choice is a function of the hold and the settler's memory, so it is part of the verdict, and two settlers choose the same seats.

Holds placed to keep others out. A hold takes places off sale until it expires, so holds that are placed and never paid can keep places from buyers. The design bounds this three ways. A party's size has a maximum, set as `venue.party.max_size`. A party holds a window at most once, because the hold's id is the party and the window. And every hold expires on its window's hold life. A buyer who makes many parties gets many holds, because the log names no one. Which people may place holds, and how many sessions one person may run, is for the storefront and its waiting room to decide, before any write exists (section 6).

4. Throughput

4.1. What is measured and what is derived

The settler is not measured. What is measured is the primitive under it, in the companion's run inside the deployment on 19 October 2026: a three node cluster, a stream with three replicas on file storage, every acknowledgement waiting for a majority (companion section 8; the method is the companion's benchmark report [12]). At p50:

Measured in the deployment	Writes in flight	p50
The local evaluation of one write	1	18.88 µs
One write end to end, the checks and the publish	1	6,042.69 µs
One write end to end, the checks and the publish	64	8,749.30 µs

The arithmetic below derives the settler's rate from these figures. It makes six assumptions, and each is named where it is used:

- **A1.** Every hub write the settler makes costs the measured end to end p50. That figure includes the benchmark task's network path to the cluster (companion section 8). The authority's own path to the cluster was not measured, and I use the figure as it is.

- **A2.** The throughput at 64 writes in flight is 64 divided by the p50, by Little's law [4], taking the p50 as the mean. The companion reports p50 and p99 for that run, not its throughput. A run whose p99 is 40,204.56 µs against a p50 of 8,749.30 µs has a mean above its median, so this estimate is high.
- **A3.** The hold stream and the reservation stream together carry the rate of the one stream measured. They are two Raft groups, so this understates them.
- **A4.** The settler's consumer receives the hold stream at least as fast as it can write. Its delivery rate was not measured.
- **A5.** The park day of section 4.4 is illustrative, chosen to size the design. The real figures of a real venue replace it.
- **A6.** The servers acknowledged every measured write before flushing it to disk, on their default interval. The guarantee needs a flush before each acknowledgement (section 3.3), and that path was not measured. Jepsen reports that a flush on every message slows a server to a few hundred messages a second [15]. If the hold path runs near that figure, every rate below falls by more than an order of magnitude, and one settler does not carry the peak of section 4.4. The rates below are for the measured setting.

4.2. The writes a hold costs

A party of n that wins costs $n + 2$ writes: the hold placed, n reservations, and the hold won. A party that loses costs 2: placed and lost. A request for a window the settler knows is full costs nothing. A conversion of a party of n costs $2n + 1$ more on the settler's streams: the hold converted, n reservations sold and n admissions. The order and its attribution edges are the connector's, on other streams, and are not counted against the settler.

4.3. One settler's rate

With one write in flight, a winning party of four is six writes in sequence: $6 \times 6{,}042.69$ µs is 36.3 ms, about 28 holds a second. A single place is three writes: 18.1 ms, about 55 a second.

With 64 writes in flight, by A2 and A3: 64 / 8,749.30 µs is about 7,315 writes a second. That is 7,315 / 6, about 1,219 winning holds of a party of four a second, or 7,315 / 3, about 2,438 single places a second.

The evaluation is not the limit. Six evaluations at 18.88 µs is 113 µs for a party of four, about 8,800 holds a second on one core. The limit is the round trip to the stream's majority, so the settler must keep many writes in flight, and it can. Writes to different places are writes to different ids, and in the fold each id's row is a function of that id's own messages, on the reservation stream as on the hold stream, so their order against one another does not matter (companion section 6.4, Lemmas 1 and 2). A hold's own path is three round trips: the placed hold, its reservations in parallel, and the won version after them. The settler waits on an acknowledgement only between two writes to the same record, and before a hold's `won`. Its decisions are made in memory in stream order, ahead of the writes they cause.

What a caller waits. A hold's own path is three writes in sequence, and the caller is answered after the third. At the measured p50 that is about 18.1 ms with one write in flight and about 26.2 ms with 64, before the settler's delivery, which A4 leaves unmeasured. With 64 in flight each write's p99 is 40.2 ms, so a hold in the tail waits more than 120 ms. A conversion adds two more writes in sequence: the hold converted, then its reservations and admissions in parallel.

4.4. A park day

Part of the day	Windows	Places
Park entry: four arrival hours, 15,000 guests each	4	60,000
Attractions: 40, open 12 hours in windows of 15 minutes, 300 places each	1,920	576,000
Screens: 12, six screenings each, 250 seats	72	18,000
Restaurants: 10, with 30 VIP tables and four seatings each	1,200	1,200
The day	about 3,200	about 655,000

The rows sum to 3,196 windows and 655,200 places; the day's figures round them to the nearest hundred and the nearest thousand, and the text uses the rounded figures.

Holds. 60,000 guests in parties of four is 15,000 parties. If a party holds eight windows a day (the park, four rides, a screening or a table, and two changes of mind), that is 120,000 holds. Counting every hold as a winner, an upper bound, they cost 120,000 × 6 = 720,000 writes, and at 7,315 writes a second that is 98 seconds of the settler's day. If every hold also converts, the conversions add 120,000 × 9 = 1,080,000 writes, and the day's total of 1,800,000 writes is 246 seconds.

The peak. Same day ride booking opens with the gates. If half the parties, 7,500, each book four rides in the first fifteen minutes, that is 30,000 holds in 900 seconds, about 33 a second. Allowing each one two retries after losing, it is 100 holds a second, or 600 writes a second at six writes a hold. On the measured setting (A6), one settler at about 1,219 holds a second has twelve times that. One write at a time, at about 28 holds a second, does not keep up, so the pipelining of section 4.3 is part of the design.

The hot window. A new ride's first window has 300 places. If 5,000 parties of four ask for it in the second it opens, 75 win and 4,925 lose. The winners cost 75 × 6 = 450 writes. The losers cost at most 4,925 × 2 = 9,850, if every request is placed before the settler knows the window is full, and nothing once it knows. The total of at most 10,300 writes is 1.4 seconds at 7,315 writes a second. So at a hot window the load is the losers, and the refusal of a known full window without a write is what bounds it. The 300 places are 300 subjects, so no one subject is hot; what is contended is the hold stream's order.

At the gates. The first arrival window admits 15,000 guests. If they arrive across its hour, that is about 4 entry scans a second, and if a third of them arrive in its first ten minutes, about 8. Each guest then scans at each window the party won, four rides and a screening or a table: five scans each, 300,000 more over twelve hours, about 7 a second. The gates write at their site (section 5), not at the hub. The companion's site wrote 2,000 records in 84 to 93 ms (companion section 10.3), a mean of 42 to 47 µs a record, on a laptop in one process; on that figure a site writes about 21,000 records a second, more than a thousand times the gate rate. The hardware of a site in a park is not the laptop, and the gate rate on it was not measured, nor was a site that flushes each write before it acknowledges it.

Storage. Taking every place as sold, each reservation has two versions, `held` and `sold`. At the companion's record of 726 bytes as the size of a version (companion section 3.2), 655,000 × 2 × 726 bytes is about 951 MB a day on each of three replicas, before the subject and headers. The holds add 120,000 × 3 × 726 bytes, about 261 MB. One admission per sold place adds 655,000 × 726 bytes, about 476 MB. The bootstrap caps every record stream it creates at 8 MiB for a tenant the test harness uses and 256 MiB for any other, and a full record stream refuses the next write, with an alarm at 80 percent of its cap (companion section 2.2). A stream that keeps days passes that cap within days, and the reservation and admission streams pass it within one, so the venue's streams take caps of their own, set from its real figures. A cap is reserved in full on every replica (companion section 11), and a write fails on the storage a broker uses, with an alarm when the fullest broker reaches 70 percent (companion section 2.2), so the venue's caps are sized against the brokers' storage as well as against its figures. A wrong cap shows first as an alarm and at worst as refused writes, never as a lost record. For the hold and reservation streams a refused write is a refused sale, and for a site's scan stream it is a refused entry, so each is sized for every record of every open window. A cap per family is this design's, beside the primitive's one rule.

4.5. Where it splits, and which split keeps the guarantee

The arithmetic says one settler carries the day above with room to spare. If a complex needs more, the guarantee and the party rules want different splits, and the primitive allows one kind. A subject has five tokens and the id is one token, so a family's stream cannot be divided inside a tenant (companion section 2.2). The only split is the tenant.

- **By party.** Two parties after the same place land on two streams, and each stream admits its own first write to the place's subject. Lemma 1 holds per stream and fails across them. This is the split that breaks the guarantee.
- **By window.** Every place is in one window, so every reservation of a place lands on one stream and the guarantee holds. But a party's day crosses about eight of some 3,200 windows, so the overlap check crosses splits on nearly every hold.

- **By attraction.** Every place is in one attraction, so the guarantee holds. A party's day crosses a handful of attractions, not thousands of windows. This is the split: a group of attractions per tenant. The overlap rule then crosses tenants, and a step in front of the attraction's settler claims the party's time on a record of its own, with the same expectation on its own subject, and releases the claim if the attraction's settler refuses. That step can refuse a party that would have fit, while a claim waits on a verdict that turns out lost. It cannot double book one.

5. The edge

5.1. The site

A site runs its own broker as a leaf of the hub, with JetStream in a domain of its own, and acknowledges its own records whether or not the hub is reachable (companion section 10.2); the site is built in the reference implementation. For each family it writes, it holds a stream of the hub stream's name and subject on one replica, and the hub's stream takes one source from it. A source copies messages across the leaf link from where it left off, so the site's records reach the hub in the site's order after the link returns. Each source is filtered to its own family's subjects under the site's tenant root, so a site's stream delivers into a hub stream only records of that stream's family and tenant, and the projector's address check refuses any record off its own address. The hub streams the site reads are mirrored at the site, current while the line is up and frozen while it is down. The leaf refuses the site's family subjects in both directions, so a record reaches the hub only by the source, once. A site is trusted for the validation of what it writes within its families, as any family writer is, and not to publish on source deliveries that are not its own (companion sections 10.2 and 10.3). A cart's machine is out in the park, and what it writes within its families is believed as written; its key reaches only its own families under the venue's root through its own source, and a revoke stops it at each server's reload (section 6).

The companion's site holds one tenant's families, those of the tenant whose root carries its worker's queues, facts and presence (companion section 10.2). A site of the venue's own tenant, and a site per cart, are part of this design. A venue has a fixed site in the park's network room, which serves every gate, ride entrance, screen door, fixed bar, kitchen and guest services desk over the park's private network. Each roving cart is a site of its own, with its own small machine, its own domain and its own key, connected over whatever the cart has and cut off whenever it rolls into a dead zone. A cart is a site that moves.

The gates, the ride entrances, the terminals and the handhelds never dial the broker; they reach their site's worker over the park's private network. A device cut off from its site's machine admits nothing and records nothing, so what keeps working while the hub is out of reach is the site, not each device. A site's streams have one replica on one machine, on the site's own disk (companion sections 10.2 and 10.3). A site that loses its disk loses what the hub has not yet sourced, and a site that loses power can lose writes it acknowledged and had not flushed; for an entry scan, that lets the ticket enter again.

Where	Families	Why
Written at a site; the hub sources it	scans, orders taken at stands, carts, tables and windows, stock moves, shift counts	They happen in the park and must be acknowledged at once, and each id is one that only one site writes
Read at a site; mirrored from the hub	spaces, windows, ticket types, admissions, vouchers, invitations, stands, menu items, offers, promotions	Gates and stands look things up locally
Only at the hub	parties, holds, reservations, and everything else	Lemma 1 needs one stream per subject, and the overlap rule needs one order per party

Why holds and reservations are hub only. The expectation of an empty subject is checked by the stream that holds the subject. A site's stream of the same name would hold its own copy of the subject, check against itself, and answer the guest before the hub could object; two sites would each sell the place. So no site has a stream bound to the reservation or hold branch, and no site worker's grant names a reservation subject. A reservation published at a site is refused at the hub as ungranted and has no stream at the site to land in.

5.2. A gate that is cut off

An admission carries a token: an Ed25519 signature [5] over the tenant, the window, the place, a serial, the reservation's epoch and the member's band, made with the venue's ticket key, which only the authority holds. The barcode or the wristband's chip carries the token. The site's worker holds the public half and verifies the signature with no lookup. It refuses an admission its mirror shows void, and a serial older than the one its mirror holds (section 3.6). It checks the window and the date in the token, and the band against the attraction's age rule from its mirror. Then it writes the entry scan.

The scan's id is over the admission, the purpose, the entry epoch and the site, and the worker publishes it expecting an empty subject on the site's own scan stream. Every gate of one site that admits to the same window writes to the same subject, so a ticket shown at two turnstiles at once enters once, with or without the hub: Lemma 1, on the site's stream. The entry epoch rises by one on each exit, so a pass out and a return are two entries.

This works for a ticket sold a minute before the cut, which the frozen mirror has never seen, because the signature is the proof and the mirror is not consulted for it. What a cut off gate cannot know is a refund made at the hub after the cut. That ticket is admitted. A place refunded on its window's day is not sold again that day (section 3.6), so the refunded ticket enters into its own place, and the window stays within its capacity. A refund made on an earlier day is sold again at once. A site cut off from that refund until the window opens can therefore admit two people for one place, and that is the one way a window exceeds its capacity. When the line returns, the check named `scans_match_admissions` lists every entry scan whose admission was voided before the scan reached the hub.

A copied token. A token is a bearer credential, and a copy enters as the original would. The entry scan admits one of them per entry epoch, so a copy shown after the original is refused as already in. A pass out raises the entry epoch, so a copy can enter while the original is outside.

Age at the gate. The token carries the band, signed. A band that needs proof and has none is verified once at the park's entry by a person looking at an identity document. The entry scan records the verification with its own id as the reference, and the guest gets the band's wristband. Ride gates on the same site read that scan from their site's own stream, with no hub. A ride gate refuses a band under the ride's rule, and a refusal is counted on the site worker's run, not written as a record.

5.3. Stands and carts

A stand's terminal sends each sale to its site's worker, which writes the order and one stock move per item drawn, at the site. A card taken while the line is down is the point of sale system's own offline mode, and its risk is that system's. A cart writes its sales, stock moves, voucher redemptions and shift count at its own site, each with the cart's position at that moment: latitude, longitude and the park's zone. The position is the cart's, never a guest's.

The hub's order, stock move, scan and shift count streams each take one source per cart. A cart that comes back after an hour in a dead zone is sourced in its own order, from where its source left off, with no effect on any other cart. In the companion's test, all of one site's records were at the hub 2.2 to 2.8 seconds after the line returned for 2,000 records, and 0.9 to 1.4 seconds for 20,000; the time is the sources re-establishing, not the transfer (companion section 10.3).

What a cart cannot promise. A voucher redeemable at a cart and at a fixed bar can be redeemed once at each while both are cut off, because they are two sites with two scan streams, and Lemma 1 holds per stream. The scan's id includes the site, so the two redemptions are two records with two ids, and the check named `scans_match_admissions` lists the double redemption by name when both are sourced. Had the id left the site out, the two would share one id, and the fold would keep one row of the two, so the read model would show one redemption where there were two. A drinks package names the stands it is good at. A cart sells alcohol only to a wristband or a document it sees, because it cannot read the park's verification scans while it has no signal.

5.4. Selling at the door while cut off

A site cannot sell places, because holds and reservations are hub only. Before opening, a hold of kind `site` takes a door allotment for the park's entry windows, settled at the hub like any other, with its reservations in the state

`allotted`. Guest services sells from that allotment in the site's own order, writing orders whose lines name the places. When they are sourced, the settler turns the allotment's reservations to `sold` and writes the admissions under the same deterministic ids. What is left after the window is released at the hub.

The allotment bounds what a cut off site can sell: no more than the places the hub gave it. Within the allotment, that each place is sold once rests on the site's one worker assigning each place once in its own order, not on the broker. Two sourced orders that name one allotted place are found by the settler, and it raises an exception for the second.

5.5. Convergence when the line returns

A hub stream that sources from sites holds the hub's own writes and each site's records, interleaved in arrival order. It is still one stream, so the companion's Lemma 1 covers it: each message has one position, and each id's row is a function of that id's messages in log order (companion section 6.4). The fold, its position guard and its tombstones carry over unchanged. The fold is at version 5, which keys every row of the read model by tenant and id, so an id that two tenants write is two rows (companion sections 2.5 and 6.4). The ids of section 2.2 are derived over the tenant, so the change moves none of them. A document's id is the hash of its bytes and carries no tenant, so a seat map or a snapshot with the same bytes under two tenants is one id and, under version 5, two rows. The companion's replay of its whole log on 20 October 2026, at fold version 4, is the evidence for the fold: no row was only in the rebuild, 45 nodes differed in their creation time alone, and the rows outside the retained log mark the boundary, that replay parity is containment of the rebuild in the live model (companion sections 6.7 and 11). The companion reports no replay at version 5. A site's record that also reached the hub another way, such as a sale reported both by the site and by the point of sale vendor's cloud, has the same id, so it lands as one row by the companion's Lemma 1, whether the hub's stream keeps one message or two (companion section 10.3).

The companion's parity test is the evidence for the shape: after a disconnection, a projector at each end folded its own streams, and in every run the two read models were equal in all five comparisons (companion section 10.3). The venue's sites are that shape with more of them.

6. What the primitive does not give

- **Order across streams.** The hold, reservation, party and order streams are separate Raft groups, and nothing orders a message on one against a message on another (companion section 6.5). That is why conversion is written on the hold's subject, and why the hold carries a copy of everything its verdict reads.
- **Order between the hub and a site.** A site's records reach the hub in arrival order when its line returns, interleaved with the hub's own writes (companion section 10.3). A refund at the hub and a scan at a cut off gate have no order between them. The check finds the pair after the fact.
- **Order by clock.** No clock decides a race. Only the stream position does.
- **Sales while the hub has no majority.** A stream's minority side cannot write (companion section 6.1). While the hub's hold or reservation stream has no majority, nothing is sold anywhere. The authority's write gets no acknowledgement, so the caller is told the hold was not placed, never that it won. A site keeps admitting, and sells only from its door allotment (section 5.4). A site cut off from the hub keeps its own order, and a device cut off from its site does nothing (section 5.1). When the majority returns, the stream resumes from what it acknowledged, and no hold was decided on the side that could not write.
- **A durable acknowledgement by default.** The broker acknowledges a write before it flushes it, unless each server is set to flush every write (section 3.3).
- **A restart from nothing.** A settler that has no snapshot reads the whole hold stream before it settles anything (section 3.2). The snapshot bounds this; a lost snapshot does not lose a verdict, it costs the full read.
- **Fairness among people.** The stream's order is the order in which writes reach its leader. Which person gets to write first is decided in front of the storefront, by a waiting room.
- **The duplicate window beyond its window.** Within two minutes of a message, the broker drops a second message with the same message id (section 3.4). Beyond that, it stores both, and the fold reduces them to one row (companion section 10.3). Neither the guarantee of section 3.3 nor the settler's recovery depends on the

window: the expectation of an empty subject holds for as long as the subject's first message is retained, and a refused settler reads the subject.

- **Retention beyond the stream's settings.** A record stream never drops a record, refuses writes when full and denies delete and purge (section 3.3). The guarantee holds on those settings, and on a cap large enough for every reservation of every open window; what the settings cannot stop is the deletion of the whole stream (section 8).
- **A split by party.** Section 4.5. The guarantee holds per stream, so every hold and reservation of a place must land on one stream, and the only split inside the primitive is the tenant.
- **A revoke ordered against a write.** A grant change reaches each server at its reload, and a write the broker accepted before it stands in the log (companion section 5). A site's worker whose key is revoked has what it wrote while cut off sourced when its line returns, because the source is the hub's, not the worker's.

7. Related work

System	What it does	Where this design differs
Ticketmaster's on sale for one 2022 tour [6]	More than 3.5 million fans registered for a presale and about 1.5 million were sent codes. The system saw 3.5 billion requests, four times its previous peak. About 15 percent of interactions had problems, including passcode validation failures that lost tickets from carts. Queues were slowed on purpose to keep the system up	The queue there protects the system from the load. Here the inventory's guarantee does not depend on the load: a lost race is a refused write, and a known full window costs no write. A waiting room is still needed, for fairness among people and to keep the storefront up, not to keep the places correct
Virtual waiting rooms [7]	A waiting room gives early arrivals random positions when a scheduled sale opens, then admits them in first in, first out order	It orders people. This design orders writes. The two compose, and this paper claims nothing about the first
Airline reservation systems: Sabre [8]	The first online reservation system linked reservation desks to one central processing centre and took 7,500 reservations an hour in the mid-1960s. It replaced hand kept cards that caused flights to be oversold and undersold	Inventory there is central, as the hub is here. An airline may sell more seats than a flight has, and United States rules govern who is denied boarding when it does [19]. A venue's licensed capacity is a hard limit, so here a second sale of a place is refused by construction, under the conditions of section 3.3, and the gate checks a signature, not a seat list
Calvin [9]	Transactions are first given one global order by a sequencer and then executed deterministically, so replicas agree on the outcome with no commit protocol	The hold stream is the sequencer and the settler is the deterministic executor, so Lemma 3 is Calvin's idea for one kind of transaction. Calvin orders general transactions across partitions. Here one stream per tenant orders one kind, and a split across tenants is the step of section 4.5, not a protocol
The escrow method [10]	Concurrent transactions decrement an aggregate quantity, such as seats left, without holding a lock on it, by reserving amounts within known bounds	A pool's places here are numbered subjects, not a counter, so a won place is a record with its own id and the double sale is refused per place. The free count is the settler's memory, derived from the stream
Linda [11]	A process withdraws a tuple from a shared space by matching a template, and two processes cannot both withdraw the same tuple	An early model for taking a place once. Here the withdrawal is the broker's publish expecting an empty subject, the space is a replicated stream that a partition's minority cannot write, and the pattern that routes the record is also the grant that permits it and the binding that stores it
The broker's key value store [13]	Creating a key publishes its first value expecting the subject empty, and the broker refuses the create if the key exists. The create succeeds again after the key is purged	This is the mechanism of Lemma 1, shipped by the broker. This design adds the place as the key, the epoch in the key so that a place is sold again under a new id rather than after a purge, the refusal of purges on the stream, and the proof
Event stores [16], [17]	An append can expect the stream not to exist, which gives optimistic concurrency per stream. A value is kept unique by deriving a stream's id	The same rule per id, and the hold's expiry is the reservation pattern's timer. Here the expectation is on a subject of a shared stream, one stream's leader

System	What it does	Where this design differs
	from it, or by reserving it first and releasing the reservation on a timer	orders every hold of the tenant, and the verdict on a party is a deterministic function of that stream
Offline ticket validation [18]	A server signs each ticket. A validator verifies the signature with the server's public key while cut off, keeps its records locally and sends them in a batch when its link returns	The gate of section 5.2 has this shape. Here the validator's records are a stream at its site that the hub sources in order, and one entry per epoch is the broker's expected empty write, not a local list
Ticketing patents [23], [24], [25], [26]	One holds ticket requests in a queue in front of the ticketing servers, in an order the server controls, with a reservation timer per step and abandoned tickets returned to sale. One sells a limited set of seats by auction: bids are ranked, ties go to the earlier bid, and seats, contiguous per bid, are assigned in a batch after the close. One reads a ticket's coded holder identity at the entrance, matches it to the holder, and carries an invalidation flag for a resold ticket in both the ticketing database and the access system's. One verifies a barcode centrally before a resale is listed, and may cancel the old barcode and issue a new one	The queue and the auction order requests and keep a group's seats together, decided by a server program, not by a stream's leader. Google Patents lists both as expired. The two access patents check over a network, against databases that must agree. Here the gate checks a signature and its site's own stream with no lookup, and a reissue is the admission's next version with a new serial (section 3.6)
Rotating mobile tickets [22]	Each ticket is tied to the holder's phone and refreshes automatically, and screenshots and recordings are blocked, so a copy does not pass	It stops a copy before the gate, and it needs the holder's phone and the seller's app. The token here is static and fits a barcode or a wristband's chip, so a copy is not stopped before the gate. It is stopped at the gate, by one entry per epoch (section 5.2)
The companion [1]	The state primitive: the five token subject, the family as a function of the schema, the stream table as the type set, the registry on the log, the fold and its proofs, the site and its measurements	This paper adds no mechanism to the primitive. It adds a family layout, one consumer and one rule, and shows that the companion's lemmas, its expectation of an empty subject and its site carry a contended domain

What is new here is narrow, and the related work above bounds it.

- Selling a key once by an expected empty write is the broker's own create.
- Keeping a value unique by deriving an id from it is event store practice.
- Deciding a group's claim with a deterministic executor over one ordered log is Calvin's.
- Checking a signed ticket offline and sending the records later is in a transit patent.

This paper adds three things, and the proofs that they hold together:

- The place, its window and its epoch form one subject, so a place is sold again under a new id and never by a purge.
- The verdict on a whole party is a function of one stream's prefix, because the facts it reads are copied onto the hold.
- A gate's one entry per epoch is the same expected empty write, on a site's stream that the hub sources in order, under one naming for the type, the route and the grant.

The broad form, that a replicated log can sell a seat once, is old.

8. Boundaries of the claim

The venue tenant is a design. The primitive under it is built and measured (companion section 8). The five token subject, the address check, the family as a function of the schema identifier, the tenant from its record, the grants and their reply prefixes, the fold, the site and its filtered sources, the family from its schema record, the record settings and alarms of the hub's streams, and the one rule for a stream's cap hold in the reference implementation. The venue's kind, its families, the settler, its routes, the ticket key, the caps per family, the record settings on its sites' streams, the venue's site worker and the venue's sites are designed on that primitive, and nothing in this paper measures them. No venue tenant runs. The rates of section 4 are derived from the companion's figures under assumptions A1 to A6, and they are only as good as those assumptions. The

measurement that replaces them is the hold stream's own round trip, at one and at 64 writes in flight, from inside the cluster, with each server flushing every write before it acknowledges it.

The guarantee needs the reservations retained. Lemma 1 holds while a reservation's first version is in the stream. Every record stream refuses a new write when full and denies delete and purge (section 3.3), so the cost of a wrong cap, or of a broker's storage filling, is refused sales, never a double one. An identity that administers streams can still delete the stream, and a stream made again holds nothing; that identity is outside the claim, as the emergency identity is in the companion.

The guarantee needs the broker to keep what it acknowledged. A first version lost after its acknowledgement, or a second leader in a split brain, admits a second winner (section 3.3). The broker's default flush does not prevent the first, and Jepsen has reported the second on nats-server 2.12.1 [15]. The check named `one_winner` finds a second winner after the fact; it does not prevent one.

The guarantee trusts one rule of one program. Lemma 1 is the broker's, but only for publishes that carry the expectation, and Lemma 2 rests on the settler releasing an epoch before it opens the next. The grants keep every member off the reservation family, and of the identities that hold the whole tree only the settler writes reservations, so the trust is in the authority's code, not in every writer's. A program with a whole tree grant that published a reservation without the header could write a second winner. The check named `one_winner` reads the stream and finds one if it exists, after the fact.

A cut off gate admits what it cannot know is void. A refund at the hub after a gate's line went down does not reach the gate until the line returns, so the ticket is admitted and listed afterwards. A refund on the window's day keeps its place off sale, so the refunded ticket enters into its own place (section 3.6). A refund made on an earlier day that a site never received, because the site was cut off until the window, is the one case in which a window can hold more people than its capacity. A voucher valid at two sites can be redeemed at each while both are cut off, and is listed afterwards. These are found, not prevented. Preventing them would need the gate to ask the hub, which is the dependence the site exists to remove.

A door allotment rests on the site's one worker. Within its allotment, a cut off site sells each place once because its worker assigns each place once, not because the broker refuses a second. The allotment bounds the damage to the places it holds, and the settler finds a double sale when the site is sourced.

One settler per tenant, one split. One consumer orders every hold of a tenant, so the tenant's hold stream is the ceiling of section 4.3. A larger complex splits by attraction across tenants and pays for the party's overlap rule with a step that can refuse a party that would have fit. A split by party is never allowed.

Fairness is outside. The stream decides among writes that reached it, in the order they reached it. Who reaches it first is decided before the write exists, and this design takes no position on it.

What sits in front of the write. The storefront decides which people may place holds, how many parties one person may make, and the order among people, before any hold exists (section 6). The overlap rule is a party's, because the log names no one and so cannot join two parties' people. The same people booked as two parties are two parties. Each needs its own park day for every member, because a ride window requires the park day (section 3.2), so booking overlapping rides that way costs a second day's ticket for each person, and each ride still admits one person per place.

What the law decides. Whether a party's record, which links to a buyer only through the processor, is personal data is for the law where the venue runs (section 2.4). The rules for accessible seats followed in section 3.6 are those of the United States [20]. Elsewhere the venue's own law sets them, and the same parts carry them. Keeping card data off the log is the connectors' work (section 2.4), and is to be verified when they are built.

A copy of the ticket key. Whoever holds the venue's private ticket key can forge tickets that a cut off site admits. The check finds them afterwards. The key's custody is the key service's, and a rotation is a new key id that the sites' mirrors carry.

References

1. Oscar Martinez, MOG Systems, Inc. *One Name for Schema, Hierarchy and Access Control on a Replicated Log.* The companion paper. https://mog.systems/papers/one-name/SPEC.html
2. nats-io. *jetstream package*, `ExpectedLastSubjSeqHeader`, `WithExpectLastSequencePerSubject`, and the stream configuration's `Discard`, `DenyDelete` and `DenyPurge`. https://pkg.go.dev/github.com/nats-io/nats.go/jetstream
3. NATS. *Streams*, the duplicate window. https://docs.nats.io/nats-concepts/jetstream/streams
4. J. D. C. Little. *A Proof for the Queuing Formula:* $L = \lambda W$. Operations Research 9(3), 1961, pp. 383-387. https://doi.org/10.1287/opre.9.3.383
5. D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang. *High-speed high-security signatures.* Journal of Cryptographic Engineering 2(2), 2012, pp. 77-89. https://doi.org/10.1007/s13389-012-0027-1
6. Ticketmaster. *Taylor Swift | The Eras Tour Onsale Explained.* 19 November 2022. https://business.ticketmaster.com/press-release/taylor-swift-the-eras-tour-onsale-explained/
7. Queue-it. *How Does Queue-it Work?* https://queue-it.com/how-does-queue-it-work/
8. IBM. *Sabre.* IBM Heritage. https://www.ibm.com/history/sabre
9. A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, D. J. Abadi. *Calvin: Fast Distributed Transactions for Partitioned Database Systems.* ACM SIGMOD 2012, pp. 1-12. https://doi.org/10.1145/2213836.2213838
10. P. E. O'Neil. *The Escrow Transactional Method.* ACM Transactions on Database Systems 11(4), 1986, pp. 405-430. https://doi.org/10.1145/7239.7265
11. D. Gelernter. *Generative Communication in Linda.* ACM Transactions on Programming Languages and Systems 7(1), 1985, pp. 80-112. https://doi.org/10.1145/2363.2433
12. MOG Systems, Inc. *primbench: the benchmark.* https://github.com/MOG-Systems-Inc/primbench. Its report, `BENCH.md`, is published beside the companion paper.
13. NATS. *ADR-8: JetStream based Key-Value Stores*, the create operation. https://github.com/nats-io/nats-architecture-and-design/blob/main/adr/ADR-8.md
14. NATS. *ADR-50: JetStream Batch Publishing.* https://github.com/nats-io/nats-architecture-and-design/blob/main/adr/ADR-50.md
15. K. Kingsbury. *NATS 2.12.1.* Jepsen, 8 December 2025. https://jepsen.io/analyses/nats-2.12.1
16. Kurrent. *Appending events*, the expected stream state. https://docs.kurrent.io/clients/dotnet/v1.0/appending-events.html
17. O. Dudycz. *How to ensure uniqueness in Event Sourcing.* 9 March 2022. https://event-driven.io/en/uniqueness-in-event-sourcing/
18. Moovel North America. *Systems and methods of providing and electronically validating tickets and tokens.* US patent 11,212,100 B2, priority 23 March 2017. https://patents.google.com/patent/US11212100B2/en
19. United States Code of Federal Regulations, 14 CFR 250.2a, *Policy regarding denied boarding.* https://www.law.cornell.edu/cfr/text/14/250.2a
20. United States Department of Justice. *ADA Requirements: Ticket Sales.* https://www.ada.gov/resources/ticket-sales/
21. PCI Security Standards Council. *PCI Data Security Standard.* https://www.pcisecuritystandards.org/standards/
22. Ticketmaster. *SafeTix.* https://business.ticketmaster.com/business-solutions/safetix/
23. Ticketmaster. *Systems and methods for queuing access to network resources.* US patent 8,463,630 B2, priority 16 December 2003. https://patents.google.com/patent/US8463630B2/en
24. Ticketmaster. *Systems for dynamically allocating finite or unique resources.* US patent 8,315,918 B1, priority 6 April 2004. https://patents.google.com/patent/US8315918B1/en
25. Ticketmaster. *Apparatus for access control and processing.* US patent 8,294,549 B2, priority 9 May 2006. https://patents.google.com/patent/US8294549B2/en
26. Live Nation Entertainment. *Networked barcode verification system.* US patent 8,510,138 B2, priority 6 March 2009. https://patents.google.com/patent/US8510138B2/en
27. Oscar Martinez, MOG Systems, Inc. *One Table: A Studio Footage Pipeline Run by One Person.* The studio paper. https://mog.systems/papers/one-table/ONE_TABLE.pdf